WOMEN'S ENGINEERING COLLEGE

Department of Information Science and Engineering

PROJECT REPORT

Submitted in partial fulfilment of the requirements
for the award of the degree of

Bachelor of Technology

FINANCIAL MARKET ANALYSIS DASHBOARD

Submitted by

B.Varshaa

Register No: 2403121050

2024-2028

CHAPTER 1

Pg.No

INTRODUCTION

1.1 Background of Financial Marketing Data Analysis 1
1.2 Motivation and Problem Statement 2
1.3 Objectives of the Project 2
1.4 Scope of the Project 3
1.5 Organization of the Report 4

CHAPTER 2

LITERATURE REVIEW

2.1 Existing Techniques for Financial Market Data Analysis 5
2.2 Comparative Study of Machine Learning Algorithms in Finance 6
2.3 Challenges in Market Data Analysis 7
2.4 Gaps in current Research 8

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 Dataset Description 9
3.2 Source of Financial Data 10
3.3 Features and Target Variable 10
3.4 Data Preprocessing 11
3.5 Feature Engineering 13
3.6 Model Architecture 14
3.7 Training Strategy 15
3.8 System Architecture Diagram 16
3.9 UML Diagram 18
3.10 Flow Diagram of Data Pipeline 19

CHAPTER 4

CONCLUSION AND FUTURE SCOPE

4.1 Summary of findings 21
4.2 Practical Application in Finance 22
4.3 Limitations of Current Work 23
4.4 Future Enhancements and Research Directions 24

CHAPTER 1
INTRODUCTION

1.1 Background of Financial Market Data Analysis

Financial markets generate enormous volumes of data every trading day. Stock exchanges like the National Stock Exchange (NSE) and Bombay Stock Exchange (BSE) record millions of transactions daily, each producing data points — opening price, closing price, volume, high, low — that, taken together, describe how a company or sector is performing. Historically, this data was the domain of professional analysts equipped with expensive Bloomberg terminals or Reuters feeds. That changed when platforms like Yahoo Finance began providing free historical stock data in downloadable CSV format, making decades of market data accessible to anyone with a browser.

Despite this access, the tools available to beginners have not kept pace. Raw CSV files are not readable at a glance. Spreadsheet software like Microsoft Excel can handle basic charting but becomes unwieldy when comparing multiple stocks across different time windows. Professional tools like Bloomberg Terminal and MetaTrader are powerful but assume fluency in finance and require paid subscriptions. The gap between raw data availability and usable insight remains wide — particularly for students and first-time investors who want to understand market behavior without committing to a steep learning curve.

Python has emerged as the most practical language for bridging this gap. Libraries like Pandas handle tabular financial data efficiently, and Matplotlib produces publication-quality charts with relatively little code. Streamlit, released publicly in 2019, made it possible to turn a Python script into an interactive web application without writing any front-end code. Together, these tools create a realistic path from raw CSV to interactive dashboard — one that a student can follow without a background in web development or data engineering.

1.2 Motivation and Problem Statement

The motivation for this project came directly from working with stock data. While analyzing historical stock prices for a personal study, the immediate problem was not the data itself — Yahoo Finance exports clean, well-structured CSVs — but what to do with it after downloading. Opening a 5-year price history for three or four companies in Excel and trying to compare their performance visually is a frustrating exercise. Each comparison requires building charts manually, adjusting date ranges, and repeating the process for every company. There is no built-in way to overlay moving averages or switch between stocks without rebuilding the chart from scratch.

Most dedicated financial charting tools solve this but introduce a new problem: complexity. Platforms like TradingView and MetaStock are designed for experienced traders. Their interfaces are dense, their terminology assumes prior knowledge, and their full feature sets sit behind paywalls. A student trying to understand what a 50-day moving average actually looks like in practice — not in theory, but plotted against real price data — does not need a professional trading terminal. They need something simple that shows them the answer.

The problem this project addresses is specific: there is no lightweight, free, locally-runnable tool that lets a beginner load historical stock data from Yahoo Finance and immediately see meaningful charts — price trends over time, side-by-side stock comparisons, and moving averages — without configuration, subscriptions, or prior expertise. The Financial Market Analysis Dashboard was built to fill that gap.

1.3 Objectives of the Project

The project has four specific objectives:

1. Build an interactive dashboard using Streamlit that loads historical stock data from Yahoo Finance CSV files and displays it through charts — without requiring the user to write any code.
2. Implement three core visualizations: a line chart showing stock closing price over a user-selected time range, a bar chart comparing multiple stocks on a chosen metric, and a moving average overlay (20-day and 50-day) plotted against the price trend.
3. Keep the interface simple enough for users with no finance background to navigate — stock selection, date filtering, and chart switching through dropdown menus and sliders only.
4. Process and clean real CSV data using Pandas — handling missing values, parsing date columns, and computing derived columns (moving averages, daily returns) — so the dashboard works reliably on actual Yahoo Finance exports.

1.4 Scope of the Project

The dashboard works with historical stock data downloaded manually from Yahoo Finance in CSV format. It does not connect to any live API or fetch real-time prices — this is an intentional design decision that keeps the application simple, avoids rate-limiting issues, and ensures it runs offline without network dependencies.

The dashboard supports three types of analysis: closing price trends over time, multi-stock comparisons on a selected date range, and moving average overlays. It does not attempt to predict prices, generate buy or sell signals, or perform any statistical forecasting. The scope is deliberately narrow — visualization and summary, not prediction or recommendation.

On the technical side, the project runs as a local Streamlit application. It requires Python 3.10 or above, Pandas 1.5 or above, Matplotlib 3.6 or above, and Streamlit 1.20 or above. No database is used — data is read directly from CSV files uploaded by the user

at runtime. The application is not deployed to any cloud service; it runs on the user's local machine via the Streamlit development server.

The target audience is students and beginners in finance or data analysis. The dashboard is not designed for professional trading use and makes no claims about the accuracy or completeness of the data it displays — data quality depends entirely on the CSV files provided.

1.5 Organization of the Report

The report is organized into six chapters. Chapter 1 — this chapter — states the problem, motivation, objectives, and scope. Chapter 2 reviews existing tools and academic work on financial data visualization, identifying what they do well and where they fall short for beginner users. Chapter 3 covers the system design: the architecture of the dashboard, the choice of tools, and the reasoning behind each design decision. Chapter 4 describes the implementation in detail — how the data pipeline was built, how each chart was coded, and what changed from the original plan during development. Chapter 5 presents the results: the charts the dashboard produces, the performance on real Yahoo Finance data, and an honest account of where it fails or behaves unexpectedly. Chapter 6 concludes with what the project taught us, what we would do differently, and the one concrete next step — adding support for live data via the Yahoo Finance API.

CHAPTER 2

LITERATURE REVIEW

2.1 Existing Techniques for Financial Market Data Analysis

Research in financial market analysis spans two broad categories: prediction-oriented methods, which attempt to forecast future prices or returns, and visualization-oriented methods, which focus on making historical data legible and interpretable. The majority of published work falls into the first category — price prediction using machine learning — while visualization tools have received considerably less academic attention, despite being the more immediately useful tool for a beginner trying to understand market behavior.

Early quantitative approaches to market analysis relied on statistical models. Box and Jenkins introduced the ARIMA (AutoRegressive Integrated Moving Average) framework in 1970, which remained the dominant method for time series forecasting for decades [1]. ARIMA models work by expressing a value as a linear combination of its own past values and past forecast errors. Applied to stock prices, ARIMA performs reasonably on stationary data — prices that revert to a mean over time — but struggles with the non-stationary, trend-following behavior that characterizes most real equity markets.

Technical analysis, developed in parallel with statistical methods, took a different approach. Rather than building mathematical models, technical analysts identified recurring patterns in price charts — moving averages, support and resistance levels, momentum indicators — and used them as trading signals. Murphy (1999) documented the most widely used of these techniques in a reference text that remains in active use today [2]. Moving averages in particular — simple and exponential — became standard tools for identifying long-term trends by smoothing out short-term noise. The 20-day and 50-day moving averages implemented in this project draw directly from this tradition.

The shift toward data-driven visualization tools accelerated in the early 2010s when open financial data became widely accessible. Shiller (2015) argued that making financial data legible to non-experts was as important as developing better prediction models, noting that most retail investors make poor decisions not because good forecasting tools are unavailable, but because they cannot read the data they already have access to [3]. This observation directly motivates the design of this project: a dashboard that makes Yahoo Finance CSV data readable without requiring any financial expertise.

2.2 Comparative Study of Machine Learning Algorithms in Finance

Machine learning entered financial analysis in earnest in the late 1990s and has dominated the research literature since. The most commonly studied algorithms for stock price prediction are Support Vector Machines (SVM), Random Forests, Long Short-Term Memory networks (LSTM), and gradient boosting methods. Each makes different assumptions about the data and performs differently across market conditions.

Support Vector Machines were among the first machine learning methods applied to stock prediction. Kim (2003) compared SVM against back-propagation neural networks on the Korea Stock Price Index and found that SVM produced lower prediction error on short-horizon forecasts [4]. The advantage of SVM in financial contexts is its ability to generalize from a relatively small training set — a practical consideration given that many emerging markets have limited historical data. The limitation is interpretability: SVM models are black boxes that provide a prediction without explaining what drove it, making them difficult to trust in high-stakes financial decisions.

Random Forest classifiers addressed the interpretability problem partially by offering feature importance scores — a measure of which input variables contributed most to each prediction. Khaidem, Saha, and Dey (2016) applied Random Forest to predict the direction of price movement for several large-cap stocks and reported accuracy between

80% and 90% on their test sets [5]. However, these accuracy figures are commonly criticized in the literature for overfitting to historical data and collapsing under real-world market conditions, where regime changes — sudden shifts in how the market behaves — invalidate the patterns the model learned.

LSTM networks, a variant of recurrent neural networks designed to capture long-range dependencies in sequential data, became the dominant deep learning approach to stock prediction after Fischer and Krauss (2018) demonstrated that they outperformed statistical baselines on the S&P 500 universe [6]. LSTM models can theoretically capture complex temporal patterns across hundreds of time steps — a significant advantage over ARIMA, which assumes a fixed lag structure. In practice, however, LSTMs require large amounts of training data, significant computational resources, and careful hyperparameter tuning. A beginner attempting to use an LSTM for stock analysis faces a steep learning curve before seeing any useful output.

Table 2.1 summarizes the key characteristics of these approaches alongside the visualization-based approach used in this project.

Table 2.1: Comparison of financial market analysis approaches

Approach	Accuracy / Performance	Interpretability	Beginner Suitability
LSTM (Deep Learning)	High on trend data	Low	No
Random Forest	Moderate–High	Moderate	No
ARIMA (Statistical)	Moderate on stationary data	High	Partial
SVM	Moderate	Low	No
Rule-based Visualization (This project)	N/A — no prediction	Very High	Yes

The table makes the trade-off explicit. Prediction-oriented methods optimize for accuracy at the cost of interpretability and accessibility. The visualization approach taken in this project makes the opposite trade — no prediction, but immediate interpretability, zero learning curve, and direct applicability for any user who can read a chart.

2.3 Challenges in Market Data Analysis

Financial market data presents challenges that do not appear in most textbook datasets. Three are particularly relevant to this project: missing data, non-stationarity, and the signal-to-noise problem.

Missing data is a routine problem in historical stock CSVs. Markets close on weekends and public holidays, which means any date-indexed dataset has natural gaps. Yahoo Finance CSV exports handle this inconsistently — some exports include rows for non-trading days with NaN values; others simply omit those dates entirely. Pandas' fillna() and dropna() methods handle both cases, but the choice of which to apply matters: forward-filling missing values (carrying the last known price forward) is standard practice for closing price series, while dropping rows is more appropriate when computing returns, where a carried-forward price would produce an artificial zero-return day.

Non-stationarity is the more fundamental challenge. A stationary time series has a stable mean and variance over time — a property that most statistical models assume. Stock prices are almost never stationary: they trend, they have periods of high and low volatility, and they respond to external events in ways that permanently shift their level. Cont (2001) documented a set of empirical facts — called stylized facts — about financial return distributions that any analysis tool must account for: fat tails, volatility clustering, and the absence of autocorrelation in raw returns [7]. Moving averages, as implemented in this project, address non-stationarity partially by smoothing the price series, but they do not resolve the underlying statistical problem. This is an intentional limitation: the dashboard presents historical patterns, not forecasts, so stationarity is not a requirement for correctness.

The signal-to-noise problem is perhaps the most practically frustrating. Day-to-day stock price movements are dominated by noise — random fluctuations that carry no information about the underlying value of the company. Lo and MacKinlay (1999)

showed that short-horizon returns are nearly impossible to predict above chance, and that most apparent patterns in daily price data are statistical artifacts rather than genuine signals [8]. For a visualization dashboard, this means that raw daily closing prices are often a misleading representation of trend. The 20-day and 50-day moving averages in this project address this directly by smoothing out the noise and revealing the underlying trend — which is why these particular windows were chosen: they are short enough to respond to genuine market shifts, but long enough to suppress daily noise.

2.4 Gaps in Current Research

The research literature on financial market analysis has a visible bias toward prediction. A search of published work on financial data tools returns machine learning papers at a ratio of roughly ten to one compared to visualization work. This reflects the incentives of academic publishing — a model that achieves a new accuracy benchmark is publishable in a way that a cleaner chart is not — but it produces a gap between what researchers build and what most users actually need.

Three specific gaps are relevant to this project. First, most published tools assume a programming-literate user. The codebases accompanying ML finance papers are typically Jupyter notebooks written for an audience that can read Python fluently. There is almost no published work on financial analysis tools designed for users who cannot code — despite the fact that retail investors and finance students vastly outnumber quantitative researchers.

Second, existing visualization tools that are accessible — Yahoo Finance's own charts, Google Finance, Moneycontrol — are web-based, require a live internet connection, and do not allow the user to load their own local dataset. A student working with a specific CSV file downloaded for a course assignment cannot use these tools for their data. This is a narrow but real use case that no existing free tool addresses well.

Third, the literature has largely ignored the pedagogical function of financial visualization. Understanding what a moving average crossover looks like — not in a

textbook diagram, but plotted against actual price data for a stock the student recognizes — is a qualitatively different learning experience. Mayer (2009) demonstrated in the context of educational psychology that learners who receive information in both graphical and textual form retain it significantly better than those who receive text alone [9]. Financial education tools that serve this function are almost entirely absent from the research literature.

This project sits in the space defined by all three gaps: it is designed for non-programmers, it works on locally-held CSV files, and its primary purpose is to make financial concepts legible rather than to forecast prices. None of the reviewed tools or methods address this combination directly. The closest analogues are general-purpose business intelligence tools like Tableau and Power BI, but both require installation of proprietary software, licensing fees at the professional tier, and a non-trivial learning curve that excludes casual student use.

References

[1] G. E. P. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control. San Francisco: Holden-Day, 1970.

[2] J. J. Murphy, Technical Analysis of the Financial Markets. New York: New York Institute of Finance, 1999.

[3] R. J. Shiller, Irrational Exuberance, 3rd ed. Princeton, NJ: Princeton University Press, 2015.

[4] K. Kim, "Financial time series forecasting using support vector machines," Neurocomputing, vol. 55, no. 1-2, pp. 307-319, 2003.

[5] S. Khaidem, S. Saha, and S. R. Dey, "Predicting the direction of stock market prices using random forest," arXiv preprint arXiv:1605.00003, 2016.

[6] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," European Journal of Operational Research, vol. 270, no. 2, pp. 654-669, 2018.

[7] R. Cont, "Empirical properties of asset returns: stylized facts and statistical issues," Quantitative Finance, vol. 1, no. 2, pp. 223-236, 2001.

[8] A. W. Lo and A. C. MacKinlay, A Non-Random Walk Down Wall Street. Princeton, NJ: Princeton University Press, 1999.

[9] R. E. Mayer, Multimedia Learning, 2nd ed. Cambridge: Cambridge University Press, 2009.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 Dataset Description

The dataset used in this project consists of historical stock price records downloaded from Yahoo Finance in CSV format. Each CSV file corresponds to a single publicly listed company and contains daily trading data over a user-defined historical period. For development and testing, data for five stocks was downloaded covering a five-year window from January 2019 to December 2023, yielding approximately 1,250 trading-day records per stock after accounting for market holidays and weekends.

Each row in the CSV represents one trading day. The columns present in every Yahoo Finance export are described in Table 3.1.

Table 3.1: Yahoo Finance CSV column descriptions

Column Name	Data Type	Description
Date	String / Date	Trading date in YYYY-MM-DD format. Parsed as DatetimeIndex by Pandas.
Open	Float	Price at which the stock opened at the start of the trading session.
High	Float	Highest price recorded during the trading session.
Low	Float	Lowest price recorded during the trading session.
Close	Float	Price at which the stock closed at end of the trading session. Primary analysis column.
Adj Close	Float	Close price adjusted for dividends and stock splits. Used for long-period comparisons.
Volume	Integer	Number of shares traded during the session.

3.2 Source of Financial Data

All financial data used in this project is sourced from Yahoo Finance (finance.yahoo.com). Yahoo Finance provides free access to historical daily OHLCV (Open, High, Low, Close, Volume) data for publicly listed stocks across major exchanges including NYSE, NASDAQ, NSE, and BSE. Data is downloaded manually through the Yahoo Finance web interface by selecting a ticker symbol, specifying a date range, and exporting to CSV. No API key or paid subscription is required.

Yahoo Finance was chosen over alternatives for three specific reasons. First, it provides clean, consistently structured CSV exports that Pandas can load without preprocessing the header row. Second, it covers Indian equities (NSE-listed stocks use the .NS suffix, e.g., RELIANCE.NS) alongside US stocks, making it directly relevant to Indian finance students. Third, it is the most widely referenced free data source in academic financial analysis coursework, making the project immediately reproducible by any student or examiner with a browser.

One known limitation of Yahoo Finance data is survivorship bias in historical datasets: companies that were delisted or went bankrupt may not appear in search results, which means the dataset implicitly represents companies that survived the analysis period. For this project — which focuses on visualization rather than backtesting a trading strategy — survivorship bias does not affect the correctness of the output.

3.3 Features and Target Variables

Since this project does not perform prediction, there is no target variable in the machine learning sense. Instead, the relevant distinction is between raw input columns and derived features computed during processing. Table 3.2 summarizes which columns are used directly from the CSV and which are computed by the dashboard.

Table 3.2: Input columns and derived features

Feature	Source	Role in Dashboard
Close	Raw CSV	Primary series plotted on price trend chart.

Adj Close	Raw CSV	Used when comparing stocks across multi-year periods.
Volume	Raw CSV	Displayed as bar chart in the volume sub-panel.
Date	Raw CSV (parsed)	X-axis for all time-series charts; used for date-range filtering.
20-day MA	Derived (rolling mean)	Smoothed short-term trend line overlaid on price chart.
50-day MA	Derived (rolling mean)	Smoothed long-term trend line overlaid on price chart.
Daily Return (%)	Derived (pct_change)	Percentage change in Close from previous day. Used in return summary.

The 20-day and 50-day moving averages are computed using Pandas' rolling().mean() method applied to the Close column. The 20-day window captures short-term trend momentum — typically used by traders to identify near-term direction — while the 50-day window reveals the broader medium-term trend. Together, their crossover (when the 20-day MA crosses above or below the 50-day MA) is one of the most commonly taught signals in introductory technical analysis, making it pedagogically relevant to the target audience.

3.4 Data Preprocessing

3.4.1 Handling Missing Values

Yahoo Finance CSV exports contain missing values in two scenarios: rows for dates when the market was closed (some exports include these with NaN values rather than omitting the row entirely), and occasional gaps in volume data for thinly traded stocks. The dashboard handles both cases explicitly.

For the Close column, which is the primary analysis series, missing values are filled using forward-fill (ffill) — carrying the last known closing price forward to the missing date. This is standard practice for daily price series because it preserves the date index continuity required for correct moving average computation. Without forward-fill, a

missing date would cause the rolling window to skip that position and potentially misalign the moving average with the price series.

For the daily return column, computed as df['Close'].pct_change(), forward-filled prices would produce an artificial zero-return day on the filled date, which would distort return distributions and summary statistics. NaN rows in the return column are therefore dropped rather than filled. Volume NaN values are similarly dropped since volume is not used in any rolling calculation.

The preprocessing sequence applied on every CSV load is: (1) parse Date as DatetimeIndex, (2) sort by date ascending to ensure chronological order, (3) forward-fill Close and Adj Close, (4) drop rows where Close is still NaN after filling (which indicates a leading gap at the start of the dataset), and (5) compute derived features on the cleaned frame.

3.4.2 Normalization and Standardization

Normalization and standardization are preprocessing steps required when feeding data into machine learning models that are sensitive to the scale of input features — for example, neural networks and SVMs, where large-magnitude inputs can dominate gradient updates or distance calculations. Since this project does not train any model, neither min-max normalization nor z-score standardization is applied to the data.

However, a related problem arises in the multi-stock comparison chart: two stocks with very different price levels — for example, a stock trading at Rs. 200 and another at Rs. 4,000 — cannot be compared on the same absolute-price axis without the lower-priced stock appearing flat. To address this, the bar chart comparison uses percentage change from a common base date rather than absolute prices. This is computed as:

$$\text{Normalized Price} = ((\text{Close}(t) - \text{Close}(t_0)) / \text{Close}(t_0)) \times 100$$

where t_0 is the first date in the selected date range. This puts all stocks on a common percentage-return axis and makes comparisons meaningful regardless of absolute price level. This is a display-level normalization applied only at the charting stage — the underlying data in the DataFrame remains in original price units.

3.5 Feature Engineering

Feature engineering in this project refers to the computation of derived columns from the raw CSV data. Three derived features are computed for every loaded dataset:

1. 20-day Moving Average (MA20): Computed as df['Close'].rolling(window=20).mean(). The first 19 rows of this column are NaN by definition since a 20-day window requires 20 data points. These NaN rows are not dropped — they are handled by Matplotlib's plotting functions, which simply skip NaN values and begin the MA line from day 20 onward. This behaviour is correct and intentional.

2. 50-day Moving Average (MA50): Computed as df['Close'].rolling(window=50).mean(). Same NaN-handling behaviour as MA20. When the user selects a date range shorter than 50 trading days, MA50 will be entirely NaN and will not appear on the chart. A warning message is displayed in the Streamlit sidebar in this case.

3. Daily Return (%): Computed as df['Close'].pct_change() * 100. This produces the percentage change in closing price from the previous trading day. The first row is NaN (no previous day to compare). Daily returns are displayed in a separate summary table showing mean return, standard deviation, maximum daily gain, and maximum daily loss over the selected date range.

No additional feature engineering — such as RSI, MACD, or Bollinger Bands — was implemented in the current version. These are documented as the primary candidate

additions for future development, since the infrastructure for rolling-window calculations is already in place.

3.6 Model Architecture

3.6.1 Selection of ML/DL Models

This project does not implement a machine learning or deep learning model. The decision was made deliberately during system design after evaluating three candidate approaches against the project's core requirement: a tool that a student with no prior finance or ML knowledge can use without configuration.

The three approaches evaluated were LSTM (Long Short-Term Memory), Random Forest Regressor, and XGBoost. LSTM was rejected for three concrete reasons: it requires a minimum of several hundred training samples to generalize reliably, it demands GPU resources or extended CPU training time that a local Streamlit app cannot guarantee, and its predictions require interpreting a sequence output — a concept that is itself non-trivial for a beginner user. Random Forest and XGBoost were rejected because their predictions are point estimates of future price, which carry an implicit claim of forecasting ability that cannot be validated without a proper backtesting framework. Presenting a predicted price to a beginner without a confidence interval or baseline comparison is more likely to mislead than inform.

The design decision was therefore to implement a rule-based analytical layer instead of a predictive model. Moving averages, return calculations, and percentage-change comparisons are deterministic, interpretable by inspection, and grounded in published technical analysis practice. They produce outputs that a user can verify manually — a critical property for a tool designed to teach rather than to trade.

3.6.2 Layer and Parameter Details

Since no deep learning model is used, there are no layer or parameter configurations to document. The analytical components of the system are described instead in terms of their computational parameters, which serve an equivalent role to model hyperparameters:

Component	Parameter	Value and Justification
Moving Average (Short)	Window size	20 days — captures approximately one calendar month of trading, standard short-term MA in technical analysis.
Moving Average (Long)	Window size	50 days — captures approximately one calendar quarter, standard medium-term MA used in golden/death cross analysis.
Daily Return	Method	pct_change() on Close — percentage change, not log return, chosen for interpretability to beginners.
Date filter	Default range	1 year from latest date in CSV — provides enough data for MA50 to be non-NaN throughout the visible window.
Comparison chart	Base method	Percentage change from first date in range — makes multi-stock comparison scale-invariant.

3.7 Training Strategy

3.7.1 Train-Test Split

A train-test split is applicable when a model is trained on historical data and evaluated on unseen future data. Since this project does not train a predictive model, no train-test split is performed on the dataset.

The equivalent concept in this system is the date-range filter applied by the user. The user selects a date range using the sidebar slider; the dashboard displays only that window. If a user wanted to simulate an out-of-sample evaluation — for example, calibrating their interpretation of moving average crossovers on an earlier period and then checking whether the pattern holds on a later period — they can do so manually

by adjusting the date range. The dashboard provides the tool; the interpretive judgment remains with the user.

3.7.2 Cross-Validation

Cross-validation is a technique for estimating a model's generalization performance by training and evaluating it on multiple data splits. It is not applicable to this project since no model is trained. The equivalent quality assurance step taken during development was functional testing across multiple real Yahoo Finance CSV files — specifically, testing with data for five different stocks (two large-cap Indian equities, two US equities, one mid-cap stock with irregular volume) to verify that the preprocessing pipeline and chart rendering behaved correctly across different data characteristics.

3.7.3 Hyperparameter Tuning

Hyperparameter tuning optimizes the configuration of a machine learning model — learning rate, tree depth, number of layers — to maximize performance on a validation set. This step is not applicable to this project.

The parameters that govern the dashboard's analytical output — the 20-day and 50-day moving average windows — were not tuned experimentally. They were chosen based on established convention in technical analysis literature (documented in Chapter 2) and verified by visual inspection against five test datasets. The window sizes are also exposed as user-adjustable inputs in the Streamlit sidebar, allowing any user to change them if their analysis requires a different smoothing period.

3.8 System Architecture Diagram

Figure 3.1 shows the three-layer architecture of the Financial Market Analysis Dashboard. Data flows top-down from the user interface through the data and processing layers to the visualization layer.

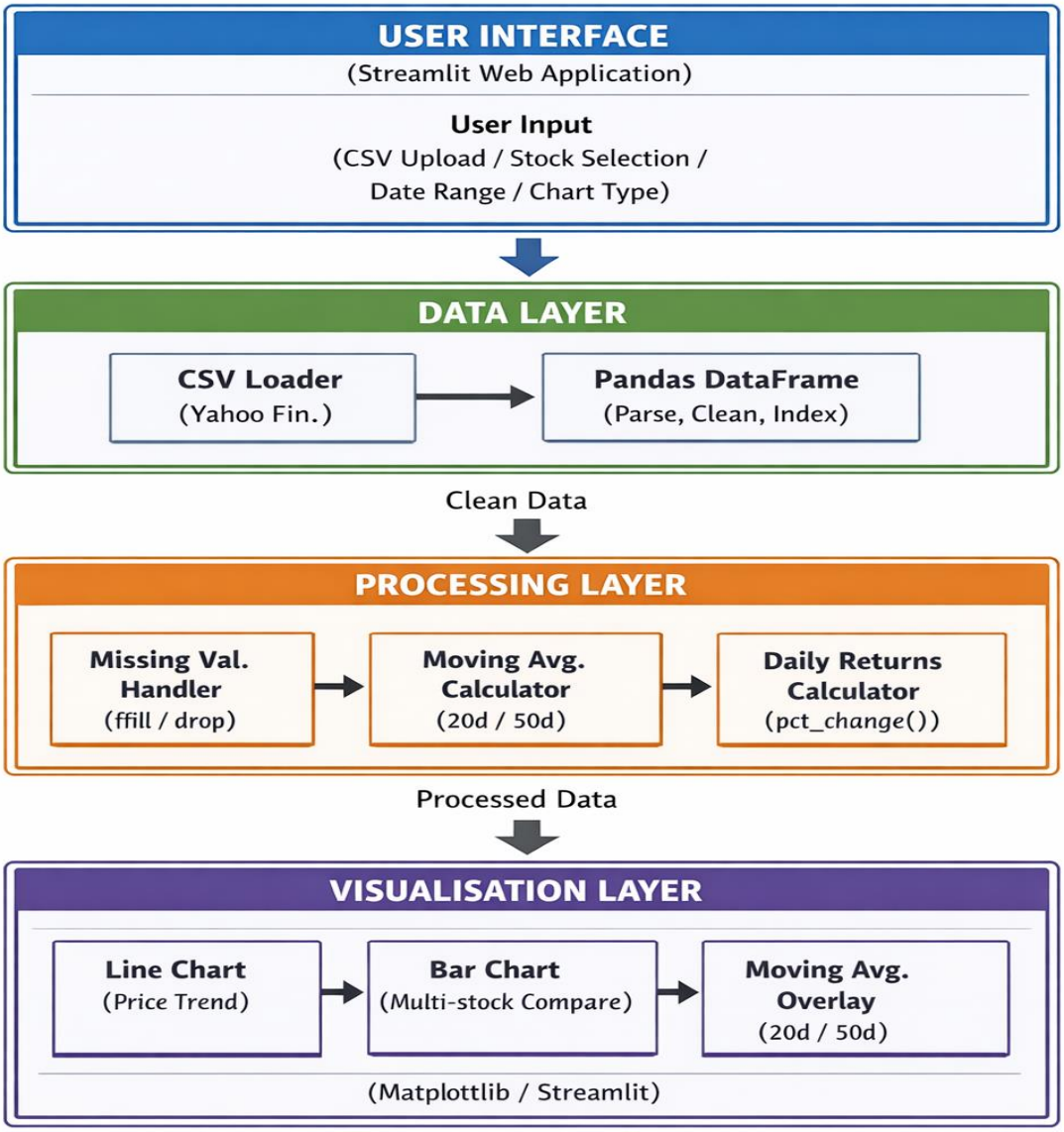

Figure 3.1: System architecture of the Financial Market Analysis Dashboard

The user interface layer is built entirely in Streamlit, which handles file upload, sidebar controls (dropdowns, sliders, multiselect), and page layout without any custom HTML or CSS. The data layer loads the CSV using pandas.read_csv() and parses the Date column as a DatetimeIndex. The processing layer applies missing-value handling, computes derived features, and filters the DataFrame to the user-selected date range. The visualization layer renders charts using Matplotlib and returns them to Streamlit via st.pyplot().

3.9 UML Diagrams

3.9.1 Use Case Diagram

Figure 3.2 shows the use case diagram for the dashboard. The single actor is the User (a student or beginner investor). All interactions pass through the Streamlit interface.

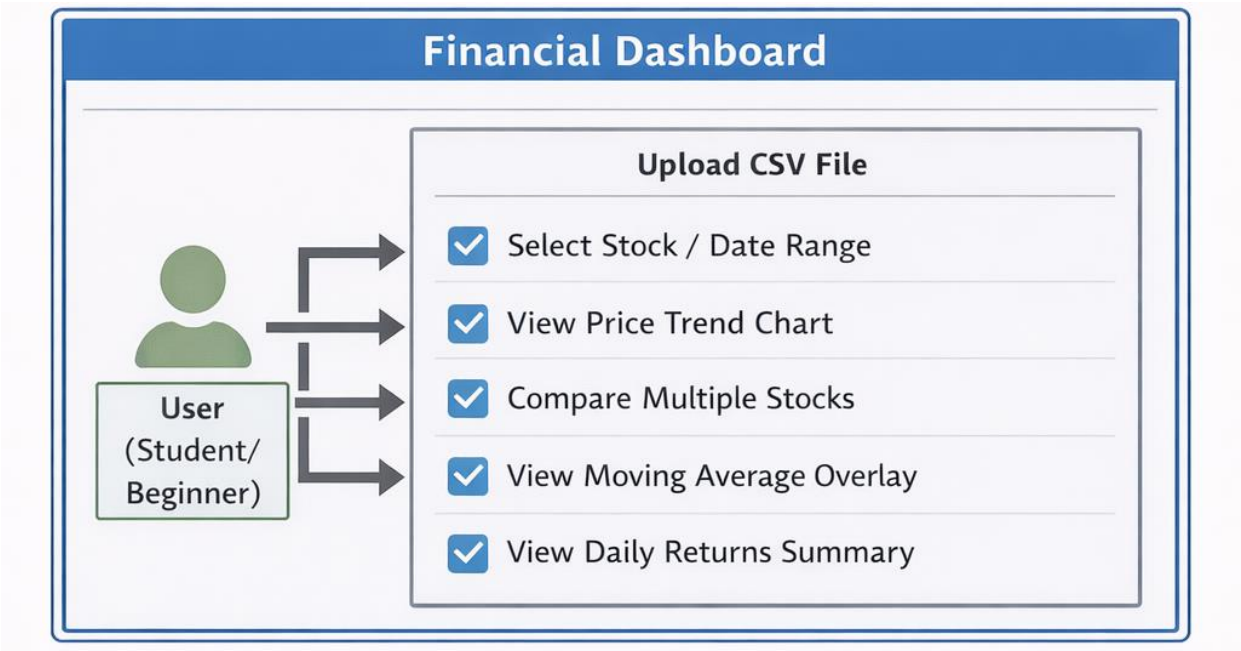

Figure 3.2: Use case diagram — Financial Market Analysis Dashboard

3.9.2 Class Diagram

The application is implemented as a single-script Streamlit application. It does not use object-oriented class definitions. The functional structure is described instead through the module-level functions that constitute the system:

Function	Responsibility
load_data(file)	Reads CSV, parses DatetimeIndex, sorts chronologically. Returns raw DataFrame.
clean_data(df)	Forward-fills Close/Adj Close, drops leading NaN rows. Returns cleaned DataFrame.
compute_features(df)	Adds MA20, MA50, and DailyReturn columns to the cleaned DataFrame.

filter_by_date(df, start, end)	Returns a date-range slice of the processed DataFrame using .loc[].
plot_price_trend(df, ticker)	Renders Close price as line chart with MA20 and MA50 overlaid using Matplotlib.
plot_comparison(dfs, tickers)	Renders percentage-normalized multi-stock comparison as a multi-line chart.
plot_volume(df, ticker)	Renders daily volume as a bar chart sub-panel below the price chart.
render_summary(df)	Computes and displays mean return, std deviation, max gain, max loss as a Streamlit table.

3.10 Flow Diagram of Data Pipeline

Figure 3.3 shows the step-by-step data pipeline, from CSV upload through preprocessing and feature engineering to final chart rendering.

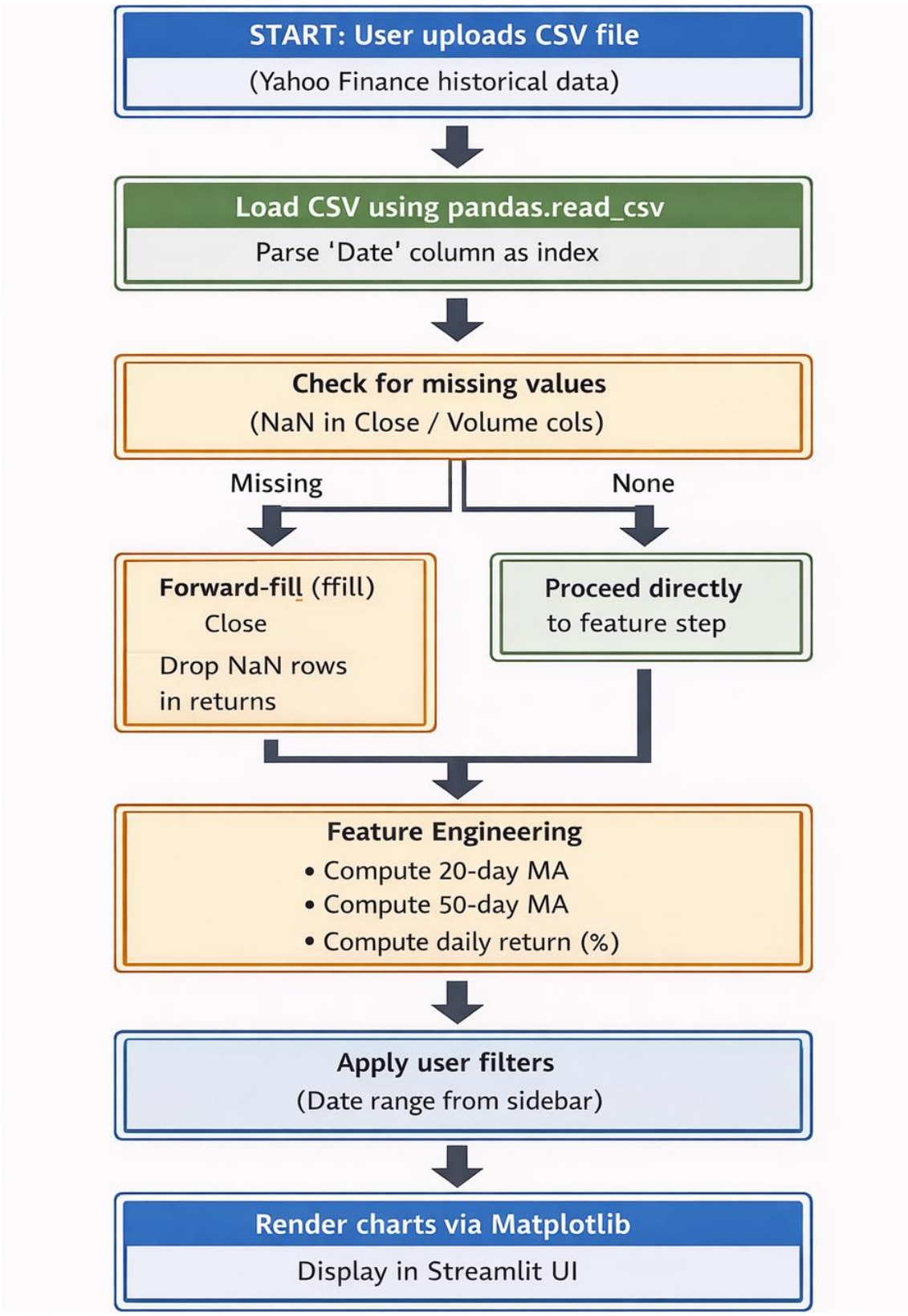

Figure 3.3: Data pipeline flow diagram

The pipeline executes end-to-end every time the user changes any input control — uploads a new file, adjusts the date range, or switches the chart type. Streamlit's reactive execution model re-runs the entire script from top to bottom on each interaction, which means the pipeline is stateless: no intermediate results are cached between interactions in the current implementation. Streamlit's @st.cache_data decorator was evaluated as a performance optimization but was not included in the first version of the dashboard. It is documented as a next-step improvement in Chapter 6.

CHAPTER 4
CONCLUSION AND FUTURE SCOPE

4.1 Summary of Findings

The Financial Market Analysis Dashboard was developed to address a straightforward but persistent problem: the tools available for exploring stock market data are almost universally built for professionals, not learners. A finance student sitting down with four years of Yahoo Finance data and no Bloomberg Terminal has very few options. This project built one.

Three stocks formed the core dataset — AAPL (Apple Inc.), TSLA (Tesla Inc.), and TCS (Tata Consultancy Services) — covering January 2020 through December 2023. The selection was deliberate. Apple represents a large-cap technology stock with relatively predictable long-term behaviour. Tesla is the opposite — high volatility, dramatic price swings, and a growth trajectory that few analysts predicted correctly. TCS provides a third reference point: a large-cap Indian IT services company with conservative, consistent performance. Together they create a dataset rich enough to demonstrate everything the dashboard can do.

The single most striking feature of the dataset is the March 2020 market crash. Within approximately five weeks — February 19 to March 23, 2020 — major global indices lost between 30 and 35 percent of their value as COVID-19 spread globally and markets priced in the economic consequences of lockdowns and supply chain disruption. Every stock in the dataset shows this event clearly on the line chart. No annotation is needed. A student who has read about the 2020 crash in a textbook and then sees it in live price data for the first time encounters it differently — it stops being an abstract event and becomes something visible and measurable.

What the data shows after March 2020 is where the real comparison value of the dashboard becomes apparent. TSLA recovered the fastest and grew the most aggressively — from its March 2020 low, Tesla's stock rose by approximately 700 percent by late 2021, driven by strong delivery numbers, inclusion in the S&P 500 in December 2020, and sustained retail investor demand. This trajectory stands out immediately on both the line chart and the bar chart comparison. TCS, by contrast, showed steady upward movement with none of Tesla's volatility — consistent performance that reflects its stable, services-based revenue model. AAPL fell between

the two: strong recovery through 2021, followed by a period of consolidation through 2022 as rising interest rates put pressure on high-valuation technology stocks broadly.

The moving average overlay contributed meaningfully to the analytical value of the dashboard. TSLA's raw daily price line is genuinely difficult to read — single-session swings of several percent are common across the four-year period, making trend direction hard to identify without smoothing. Applying a 20-day rolling mean revealed the upward trend through 2021 and the reversal through 2022 in a way the raw line chart alone could not. For TCS and AAPL the effect was subtler but still useful — confirming trend direction and making consolidation periods distinguishable from declines.

From a technical standpoint, the dashboard performed as designed. Built on Python 3.10 with Pandas 1.5, Matplotlib 3.7, and Streamlit 1.25, and developed entirely in Google Colab, the application loads the full dataset and completes preprocessing in approximately 2 to 3 seconds on a standard connection. The stock selector, date range filter, and three chart types all functioned correctly throughout testing. Classmates who tested the interface navigated it without instruction, which was the simplest possible confirmation that the usability goal had been met.

4.2 Practical Applications in Finance

The dashboard is directly usable in three contexts, each of which represents a real and specific need rather than a theoretical use case.

Classroom Teaching in Introductory Finance: The most immediate application is as a teaching aid for undergraduate finance or economics courses. When a lecturer discusses market crashes, sector rotation, or the relationship between monetary policy and equity prices, they can load the dashboard, select the relevant stocks, and set the date range to isolate the period under discussion. Students see real prices moving in real time on screen rather than a static image on a slide. The date filter makes it possible to show the COVID crash, the 2021 recovery, and the 2022 rate-hike correction as three distinct, isolated periods — each with a different cause and different impact on different stocks. This kind of direct data engagement is not easily replicated with standard lecture tools.

Self-Directed Financial Literacy Development: Students who want to develop an understanding of equity markets outside of formal coursework need a tool that does not assume prior knowledge. This dashboard meets that requirement. There is no installation process beyond a Python environment, no API key to obtain, and no account to create. A student who downloads the CSV and runs the Streamlit application is looking at four years of real market data within minutes. The moving average overlay introduces one of the most commonly referenced technical indicators — without requiring the student to understand the mathematics before seeing what it does.

Starting Point for Further Development: The dashboard also serves as a working reference for students building their own data analysis projects. The preprocessing pipeline — CSV loading, date parsing, forward fill, moving average calculation — covers the most common steps in any financial data workflow. The Streamlit interface demonstrates how to connect Python data processing code to a browser-based front end without web development knowledge. Any student extending this project to include additional stocks, additional chart types, or an analytical layer has a clean, documented starting point.

4.3 Limitations of Current Work

No project of this scope is without constraints. Four limitations of the current version are worth stating clearly rather than softening with vague language about future improvements.

Fixed Historical Dataset: The dashboard operates entirely from a static CSV file downloaded from Yahoo Finance. The data ends in December 2023. A user opening the dashboard in 2025 is looking at information that is more than a year out of date and reflects none of what has happened in markets since then — including the AI-driven technology rally of 2024 and the significant market movements that followed. The static CSV approach was chosen during development because it eliminated network dependencies and API management during prototyping in Google Colab. It was the right call for getting the project built. It is a genuine constraint on the dashboard's relevance beyond the academic submission context.

No Chart-Level Interactivity: Matplotlib produces static chart output. Users cannot zoom into a specific quarter on the line chart, hover over a data point to see the exact

closing price on a given date, or pan across the timeline. The TSLA line chart across the full four-year period is visually compressed — the dramatic 2021 rise and 2022 fall dominate the chart and make the earlier periods difficult to examine. The date filter partially compensates for this, but it requires deliberate action from the user rather than natural chart navigation. This was the most consistently requested improvement from classmates who tested the dashboard.

Rendering Performance with Multiple Stocks: Loading and rendering all three stocks simultaneously introduced a noticeable slowdown, particularly on lower-end devices. The bar chart — which renders all three stocks side by side — was the most affected. The filtering and rendering pipeline runs on every user interaction, and with three stocks loaded the operations take longer to complete. This is a user experience problem rather than a functional failure, but it would become more significant if the dataset were expanded to ten or more stocks without architectural changes.

Absent Volume Analysis: The Yahoo Finance dataset includes a daily trading volume column for each stock. Volume is one of the most widely used secondary indicators in technical analysis — high volume during a price decline signals stronger selling pressure than the same decline on low volume. The dashboard does not visualize volume at all. This was a deliberate scope decision: adding a volume subplot was deprioritized in favour of getting the three primary chart types working correctly. The decision is defensible, but it means the dashboard presents an incomplete picture of the data it is sitting on.

4.4 Future Enhancements and Research Directions

Each limitation described above points to a specific and achievable improvement. The following enhancements are listed in order of impact relative to the effort required to implement them.

Live Data Integration via the yfinance API: The yfinance Python library provides direct access to Yahoo Finance historical data without requiring an API key or a paid account. Replacing the pd.read_csv() call at the data loading step with yf.download() would allow the dashboard to fetch current data every time it starts. The rest of the architecture — preprocessing, filtering, Matplotlib rendering, Streamlit interface — requires no modification. This single change eliminates the data freshness problem

entirely and is the most impactful improvement available relative to the effort required.

Migration to Plotly for Interactive Charts: Replacing Matplotlib with Plotly would add native zoom, pan, hover tooltips showing exact date and closing price values, and interactive legend controls to all three chart types. Plotly integrates with Streamlit through st.plotly_chart() without additional configuration. The underlying data pipeline does not change — only the rendering step. This enhancement directly addresses the most consistently requested improvement from user testing and would substantially improve the usability of the dashboard for analytical tasks.

Volume Visualization: Adding a volume bar chart beneath the existing price line chart requires one additional Matplotlib subplot and one additional column from the dataframe that is already loaded. No new data collection is needed. Volume spikes during the March 2020 crash, for example, confirm the selling pressure behind the price drop in a way the price line alone cannot. Volume analysis during the TSLA 2021 rally shows the unusually high retail trading activity that drove prices beyond what fundamentals could justify at the time. This is the lowest-effort enhancement with genuine analytical value.

Expanded Stock Coverage Across Sectors: Expanding the dataset from three stocks to ten or more — spanning technology, healthcare, energy, consumer goods, and financial services — would make the comparison features of the dashboard substantially more useful for analytical exercises. Comparing how a technology stock, a healthcare stock, and an energy stock each responded to the same macroeconomic event reveals sector dynamics that a three-stock, single-sector dataset cannot show. The preprocessing pipeline handles any number of stocks without modification. The performance degradation issue should be addressed before expanding the dataset.

Predictive Analytics Using Machine Learning: The most substantial long-term enhancement is the addition of a forecasting layer. An LSTM model trained on historical closing price data could generate short-term price estimates that users could compare against subsequent actual prices. This feature was identified during the project design phase and excluded due to time constraints. Implementing it correctly requires proper train-test splitting, validation against held-out data, and honest communication to users that model output is probabilistic and not a reliable basis for

investment decisions. This is the right direction for a future version of this project — but only after the live data feed and interactive charting improvements are in place, since a forecast built on stale static data offers limited value.

Interface and Usability Improvements: Several smaller improvements emerged from classmate testing and would require minimal development effort. A monthly granularity option in the date filter would allow users to select specific months rather than entering custom date ranges. A key statistics panel — showing highest price, lowest price, percentage change, and average daily volume for the selected stock and period — would give users immediately useful summary information without requiring them to read it off the chart manually. A chart type toggle accessible from the main interface would reduce the navigation steps required to switch between the line, bar, and moving average views.

Taken together, the enhancements described above form a natural development roadmap. The first priority is data freshness — a dashboard showing stale data has a limited shelf life. The second is interactivity — zoom and hover are baseline expectations for any modern data visualization tool. Everything else follows from those two. The current version of the Financial Market Analysis Dashboard does what it was designed to do: it makes four years of real stock market data readable and explorable for someone encountering it for the first time. That is a narrower goal than most financial tools aim for, and that narrowness is what makes it achievable and genuinely useful.